Security Assessment Report

Target: D:\test | Model: deepseek-r1:7b | Risk: CRITICAL

Scan Metadata

Target	D:\test
Timestamp	2026-06-03T10:58:07.192661+00:00
Files Scanned	1
Code Files	1
Languages	PHP
AI Model	deepseek-r1:7b
Ollama URL	http://localhost:11434
Total Findings	21

Risk Summary

CRITICAL	HIGH	MEDIUM	LOW
1	15	4	1

21 AI Findings | **0** Dependency CVEs

Executive Summary

Assessment of **D:\test** using deepseek-r1:7b via Ollama. 1 code file(s) (PHP) were analysed in full — the complete source of each file was sent to the AI for semantic reasoning. Overall risk posture: **CRITICAL**.

Detailed Findings (21)

#	Vulnerability	Severity	Score	File	CWE	Conf
1	Dynamic Function Call	CRITICAL	9.0	index.php	CWE-13	medium
2	Command Injection	HIGH	8.0	index.php	CAPEC-025	medium
3	XSS - Echoed User Input	HIGH	8.0	index.php	XSS	medium
4	SQL Injection	HIGH	8.0	index.php	SQL Injection	medium
5	File Inclusion - Command Injection	HIGH	8.0	index.php	Arbitrary Command Execution	medium
6	XSS	HIGH	7.5	index.php	CWE-289	medium
7	SQL Injection	HIGH	7.5	index.php	CWE-246	medium
8	CSQI	HIGH	7.5	index.php	CWE-127	medium
9	PHP Injection	HIGH	7.5	index.php	CWE-237	medium
10	Missing CSRF Protection	HIGH	7.5	index.php	CWE-315	medium
11	Code Injection via eval	HIGH	7.5	index.php	CWE-254	medium
12	Object Injection	HIGH	7.5	index.php	CWE-259	medium
13	MailInjection	HIGH	7.5	index.php	CWE-267	medium
14	File Upload Vulnerability	HIGH	7.5	index.php	CWE-279	medium
15	File Deletion Vulnerability	HIGH	7.5	index.php	CWE-17	medium
16	Directory Listing	HIGH	7.5	index.php	CWE-12	medium
17	Insecure Randomness	MEDIUM	5.0	index.php	CWE-358	medium
18	Process Information Exposure	MEDIUM	5.0	index.php	CWE-345	medium
19	Race Condition	MEDIUM	5.0	index.php	CWE-279	medium
20	Insecure File Permissions	MEDIUM	5.0	index.php	CWE-163	medium
21	Hardcoded Password Check	LOW	2.5	index.php	CWE-349	medium

Finding Details

[CRITICAL] Dynamic Function Call #1

File	D:\test\index.php
Lines	1 – 1
Language	PHP
CWE	CWE-13
CVE	N/A
Confidence	medium
Source	AI

Attack Path

Entry: $_GET['func'] | Impact: Arbitrary code execution on the server side | Likelihood: High

Prerequisites: ($_GET['func'] is a malicious function)

Remediation

Sanitize $_GET['func'] to only allow predefined functions or validate its contents.

Code Snippet

```
<?php
```

[HIGH] Command Injection #2

File	D:\test\index.php
Lines	81 – 81
Language	PHP
CWE	CAPEC-025
CVE	N/A
Confidence	medium
Source	AI

Explanation

The `shell_exec` function is used to execute arbitrary commands based on user input.

Attack Path

Entry: User input via $user_input | Impact: Arbitrary command execution by attacker | Likelihood: High

Prerequisites: shelling out command

Remediation

Avoid using `shell_exec` and use parameterized queries instead.

Code Snippet

```
shell_exec("grep " . $user_input . " /var/log/auth.log");
```

[HIGH] XSS - Echoed User Input #3

File	D:\test\index.php
Lines	72 – 72
Language	PHP
CWE	XSS
CVE	N/A
Confidence	medium
Source	AI

Explanation
User input is echoed without proper escaping, allowing XSS attacks.

Attack Path
Entry: Echo statement in line 72 | Impact: Information disclosure by attacker | Likelihood: High

Prerequisites: Unescaped user input

Remediation
Escape HTML characters and use proper sanitization.

Code Snippet
```
echo "<input type='text' value='" . $user_input . "'>";
```

[HIGH] SQL Injection #4

File	D:\test\index.php
Lines	56 – 56
Language	PHP
CWE	SQL Injection
CVE	N/A
Confidence	medium
Source	AI

Explanation
The WHERE clause in SQL query uses a hardcoded value, allowing arbitrary queries.

Attack Path
Entry: SQL query on line 56 | Impact: Arbitrary database queries by attacker | Likelihood: High

Prerequisites: Hardcoded WHERE clause

Remediation
Use parameterized queries and bind parameters.

Code Snippet
```
$result = $conn->query($sql);
```

[HIGH] File Inclusion - Command Injection #5

File	D:\test\index.php
Lines	82 – 83
Language	PHP
CWE	Arbitrary Command Execution
CVE	N/A
Confidence	medium
Source	AI

Explanation

The `exec` and `passthru` functions execute shell commands based on user input.

Attack Path

Entry: User input via $file_name | Impact: Arbitrary command execution by attacker | Likelihood: High

Prerequisites: Unescaped user input

Remediation

Avoid using `exec` and `passthru`, use parameterized queries instead.

Code Snippet

```
exec("ls -la " . $file_name); passthru("cat " . $file_name);
```

[HIGH] XSS #6

File	D:\test\index.php
Lines	117 – 118
Language	PHP
CWE	CWE-289
CVE	N/A
Confidence	medium
Source	AI

Explanation

The code uses simplexml_load_string and DOMDocument to parse user input, which can lead to XSS vulnerabilities.

Attack Path

Entry: User-controlled input passed to simplexml_load_string | Impact: Exposes user input as raw XML content that could be manipulated by attackers. | Likelihood: High

Prerequisites: Untrusted XML data

Remediation

Sanitize the user input before parsing and use proper escaping.

Code Snippet

```
$xml = simplexml_load_string($user_input); $dom = new DOMDocument();
```

[HIGH] SQL Injection #7

File	D:\test\index.php
Lines	190 – 190
Language	PHP
CWE	CWE-246
CVE	N/A
Confidence	medium
Source	AI

Explanation
The code uses mysqli::multi_query with user-controlled SQL queries, increasing the risk of SQL Injection.

Attack Path
Entry: User-controlled SQL statements in multi_query | Impact: Attacker can execute arbitrary SQL commands. | Likelihood: High

Prerequisites: Untrusted SQL input

Remediation
Use prepared statements and escape user input.

Code Snippet
```
$conn->multi_query("SELECT * FROM users; " . $_GET['sql2']);
```

[HIGH] CSQI #8

File	D:\test\index.php
Lines	126 – 127
Language	PHP
CWE	CWE-127
CVE	N/A
Confidence	medium
Source	AI

Explanation
The code uses a query parameter in the MongoDB find method, which can lead to Cross-Site Query Injection.

Attack Path
Entry: User-controlled query string | Impact: Attacker can inject malicious queries into the database. | Likelihood: High

Prerequisites: Untrusted query parameters

Remediation
Sanitize and escape all user input when using it in SQL queries.

Code Snippet
```
$collection = (new MongoDB\Client)->test->users; $result = $collection->find(['username' => $user_input]);
```

[HIGH] PHP Injection #9

File	D:\test\index.php
Lines	130 – 131
Language	PHP
CWE	CWE-237
CVE	N/A
Confidence	medium
Source	AI

Explanation
The code uses a user-controlled variable in the render_template method, which can lead to Cross-Site PHP Injection.

Attack Path
Entry: User-controlled variable passed to render_template | Impact: Attacker can inject malicious PHP code into web pages. | Likelihood: High

Prerequisites: Untrusted variables

Remediation
Sanitize and escape all user input.

Code Snippet
```
$twig = new Twig\Environment(new Twig\Loader\ArrayLoader()); echo $twig->render("Hello {{ name }}", ['name' => $user_input]);
```

[HIGH] Missing CSRF Protection #10

File	D:\test\index.php
Lines	170 – 173
Language	PHP
CWE	CWE-315
CVE	N/A
Confidence	medium
Source	AI

Explanation
The code does not implement CSRF protection when submitting form data, making it vulnerable to CSRF attacks.

Attack Path
Entry: Form submission without CSRF tokens | Impact: Attacker can manipulate web pages by forging requests from different domains. | Likelihood: High

Prerequisites: Untrusted form data

Remediation
Implement CSRF protection in all form submissions.

Code Snippet
```
// Missing CSRF token if ($_POST['action'] == 'delete_user') { $conn->query("DELETE FROM users WHERE id = " . $_POST['user_id']); }
```

[HIGH] Code Injection via eval #11

File	D:\test\index.php
Lines	102 – 102
Language	PHP
CWE	CWE-254
CVE	N/A
Confidence	medium
Source	AI

Explanation

The code uses eval to execute PHP statements, which can lead to Code Injection attacks if user-controlled code is executed.

Attack Path

Entry: eval() function | Impact: Attacker can inject arbitrary commands into the application. | Likelihood: High

Prerequisites: Untrusted code in eval

Remediation

Avoid using eval and use prepared statements instead.

Code Snippet

```
$calc_result = calculate($user_input);
```

[HIGH] Object Injection #12

File	D:\test\index.php
Lines	105 – 108
Language	PHP
CWE	CWE-259
CVE	N/A
Confidence	medium
Source	AI

Explanation

The code uses unserialize to create objects from user-controlled data, which can lead to Object Injection attacks.

Attack Path

Entry: Untrusted data passed to unserialize | Impact: Attacker can inject malicious object properties into the application. | Likelihood: High

Prerequisites: Untrusted serialized data

Remediation

Sanitize and escape all user-controlled data.

Code Snippet

```
$unserialized_data = processData($serialized); // PHP Object Injection $object =
unserialize(base64_decode($serialized));
```

[HIGH] MailInjection #13

File	D:\test\index.php
Lines	137 – 137
Language	PHP
CWE	CWE-267
CVE	N/A
Confidence	medium
Source	AI

Explanation
The code sends emails with user-controlled content, which can lead to Email Injection attacks.

Attack Path
Entry: User-controlled email body and subject | Impact: Attacker can send unsolicited emails with malicious content. | Likelihood: High

Prerequisites: Untrusted email content

Remediation
Do not send emails from untrusted sources.

Code Snippet
```
mail("admin@example.com", "Subject", $user_input, "From: " . $user_input);
```

[HIGH] File Upload Vulnerability #14

File	D:\test\index.php
Lines	140 – 142
Language	PHP
CWE	CWE-279
CVE	N/A
Confidence	medium
Source	AI

Explanation
The code allows file uploads without proper validation, increasing the risk of malicious file inclusion.

Attack Path
Entry: $_FILES['upload']['name'] variable | Impact: Attacker can rename files to anything they want. | Likelihood: High

Prerequisites: Untrusted file names

Remediation
Validate and sanitize file names before processing.

Code Snippet
```
if ($_FILES['upload']['error'] == UPLOAD_ERR_OK) { move_uploaded_file($_FILES['upload']['tmp_name'],
"/uploads/" . $_FILES['upload']['name']); }
```

[HIGH] File Deletion Vulnerability #15

File	D:\test\index.php
Lines	1 – 1
Language	PHP
CWE	CWE-17
CVE	N/A
Confidence	medium
Source	AI

Attack Path

Entry: $_GET['file_to_delete'] | Impact: Deletes potentially harmful files | Likelihood: High

Prerequisites: ($_GET['file_to_delete'] contains malicious filenames)

Remediation

Sanitize $_GET['file_to_delete'] to prevent deletion of unauthorized files.

Code Snippet

```
<?php
```

[HIGH] Directory Listing #16

File	D:\test\index.php
Lines	1 – 1
Language	PHP
CWE	CWE-12
CVE	N/A
Confidence	medium
Source	AI

Attack Path

Entry: $_GET['list'] | Impact: Enables directory enumeration | Likelihood: High

Prerequisites: ($_GET['list'] is 'true')

Remediation

Sanitize $_GET['list'] to prevent directory listing.

Code Snippet

```
<?php
```

[MEDIUM] Insecure Randomness #17

File	D:\test\index.php
Lines	149 – 150
Language	PHP
CWE	CWE-358
CVE	N/A
Confidence	medium
Source	AI

Explanation
The code generates weak random numbers for token and password creation, which can be predictable.

Attack Path
Entry: rand() and mt_rand() functions | Impact: Predictable tokens can lead to replay attacks or brute force attempts. | Likelihood: Medium

Prerequisites: Unpredictability of tokens

Remediation
Use a cryptographically secure random number generator.

Code Snippet
```
$reset_token = rand(100000, 999999); $password = mt_rand();
```

[MEDIUM] Process Information Exposure #18

File	D:\test\index.php
Lines	153 – 155
Language	PHP
CWE	CWE-345
CVE	N/A
Confidence	medium
Source	AI

Explanation
The code calls phpinfo() when the 'info' parameter is present in GET requests, exposing process information.

Attack Path
Entry: phpinfo() function | Impact: Attacker can gain access to sensitive system information. | Likelihood: Medium

Prerequisites: Presence of 'info' parameter

Remediation
Do not call phpinfo() when processing GET requests.

Code Snippet
```
if (isset($_GET['info'])) { phpinfo(); }
```

[MEDIUM] Race Condition #19

File	D:\test\index.php
Lines	176 – 180
Language	PHP
CWE	CWE-279
CVE	N/A
Confidence	medium
Source	AI

Explanation
The code uses file locking with read/write permissions, which can lead to race conditions if another process accesses the same file.

Attack Path
Entry: Forking a PHP process | Impact: Race condition may allow unauthorized access or data corruption. | Likelihood: Medium

Prerequisites: File lock conflict

Remediation
Use proper locking mechanisms and handle file locks carefully.

Code Snippet
```
$file_lock = fopen("/tmp/counter.txt", "r+"); $count = fread($file_lock, filesize("/tmp/counter.txt"));
$count++; file_put_contents("/tmp/counter.txt", $count);
```

[MEDIUM] Insecure File Permissions #20

File	D:\test\index.php
Lines	187 – 187
Language	PHP
CWE	CWE-163
CVE	N/A
Confidence	medium
Source	AI

Explanation
The code uses chmod with 0777, which may not cover all necessary files and increases the risk of file overwrite.

Attack Path
Entry: Chmod command | Impact: Attacker can overwrite important system files or directories. | Likelihood: Medium

Prerequisites: File permissions

Remediation
Use more granular and specific file permissions.

Code Snippet
```
chmod("/var/www/config.inc.php", 0777);
```

[LOW] Hardcoded Password Check #21

File	D:\test\index.php
Lines	182 – 184
Language	PHP
CWE	CWE-349
CVE	N/A
Confidence	medium
Source	AI

Explanation

The code checks the password against 'admin123' without proper authentication, making it easy for attackers to bypass.

Attack Path

Entry: Password check | Impact: Attacker can easily gain access by providing a hardcoded password. | Likelihood: Low

Prerequisites: Hardcoded password

Remediation

Remove hardcoded passwords and implement proper authentication.

Code Snippet

```
if ($_POST['password'] == 'admin123') { echo "Access granted"; }
```